

Class02B: BP Accelerated Learning

Why do we need to sum the squared errors?

$$E = \frac{1}{2} \sum_{j=1}^q (c_j^k - c_j)^2$$

The sum of the squared errors is a useful indicator of the network's performance especially for **regression** problem. The BP training algorithm attempts to minimize this criterion.

Accelerated Learning in Multilayer Neural Network

Method 1: To include a momentum term in the delta rule.

Between hidden and output layer:

$$\Delta w_{ij}(p) = \alpha \cdot b_i(p) \cdot d_j(p) + \beta \cdot \Delta w_{ij}(p-1)$$

where b_i is the output of neuron i in the hidden layer

$d_j(p)$ is the error gradient at neuron j in the output layer

$$d_j(p) = (c_j^d(p) - c_j(p))$$

Between input and hidden layer:

$$\Delta v_{hi}(p) = \alpha \cdot a_h(p) \cdot e_i(p) + \beta \cdot \Delta v_{hi}(p-1)$$

where a_h is the output of neuron h in the input layer

$e_i(p)$ is the error gradient at neuron i in the hidden layer

Typically, the momentum constant β is set to 0.95.

The above equation is called the **Generalized Delta Rule**.

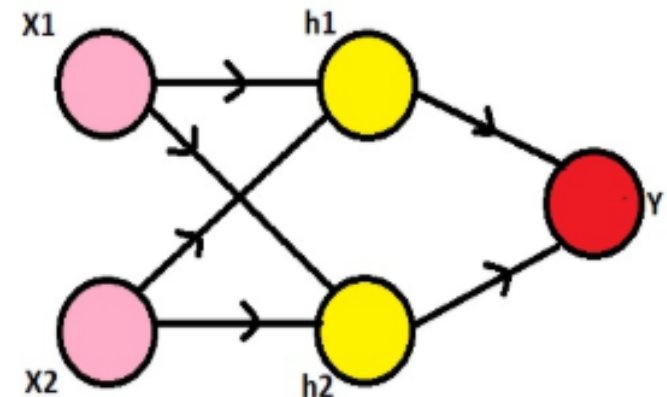
Why do we need the momentum constant?



- The inclusion of momentum in the BP algorithm has a stabilising effect on training.
- It tends **to accelerate descent in the steady downhill direction**, and to **slow down the process when the learning surface exhibits peaks and valleys**.
- In the Exclusive-OR example, the number of epochs is reduced from 224 to 126.

The following is the Truth table for XOr function

Input 1	Input 2	Output
0	0	0
0	1	1
1	0	1
1	1	0



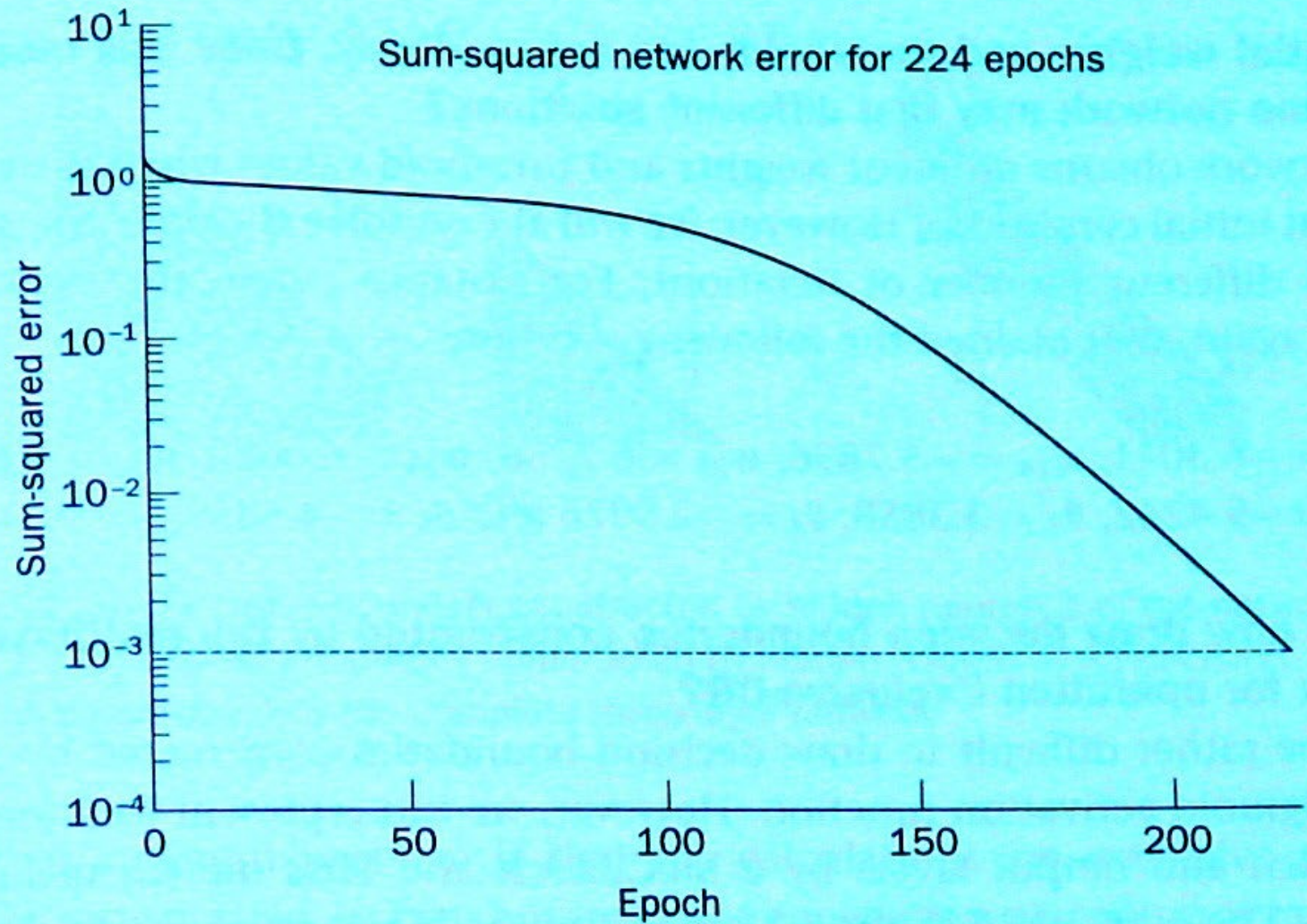
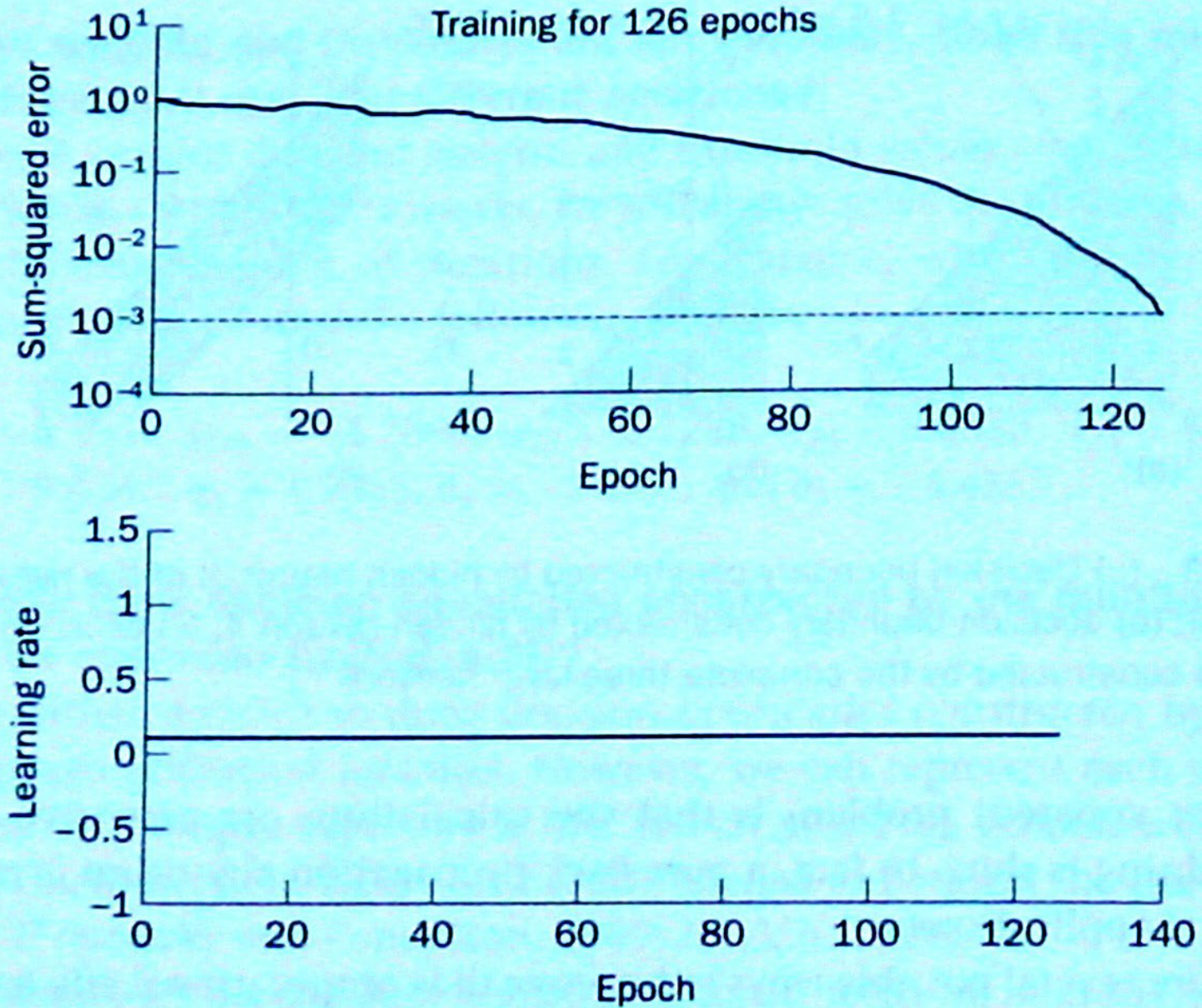


Figure 6.11 Learning curve for operation Exclusive-OR

Learning with momentum



Method 2: Adaptive Learning Rate

To accelerate the convergence of BP learning, we can adjust the learning rate parameter α during training.

Small α means small changes to the weights and leads to the smooth learning curve.

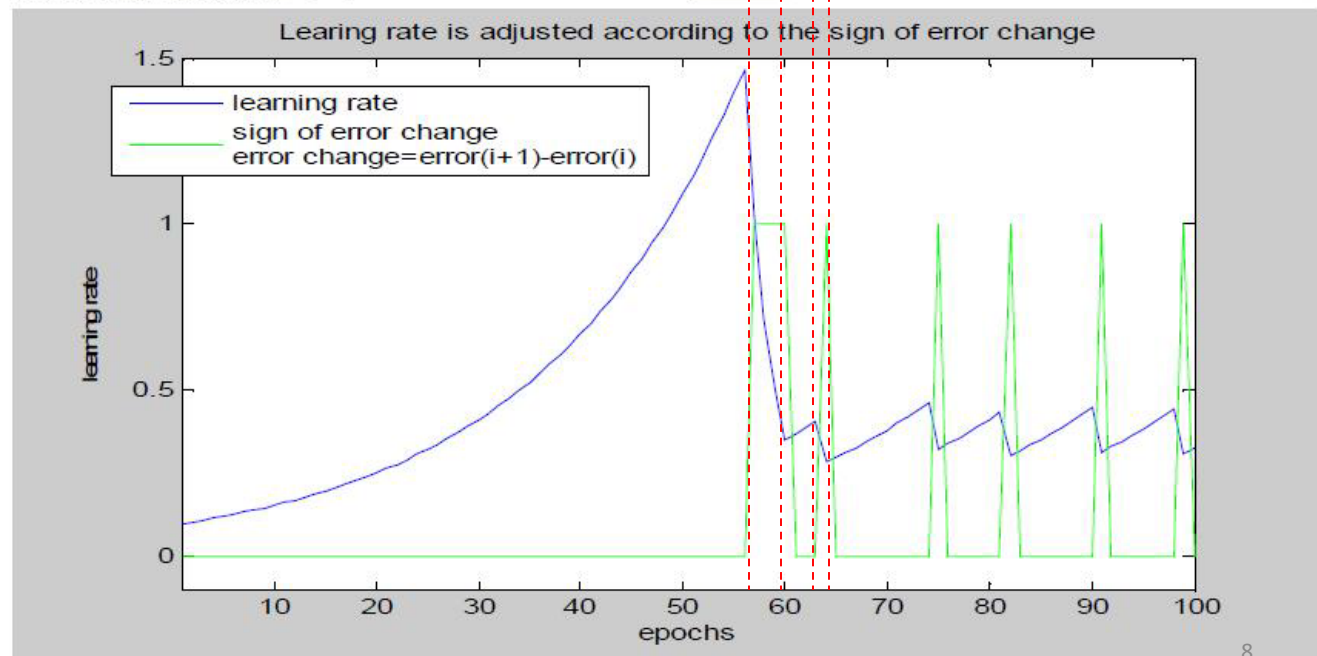
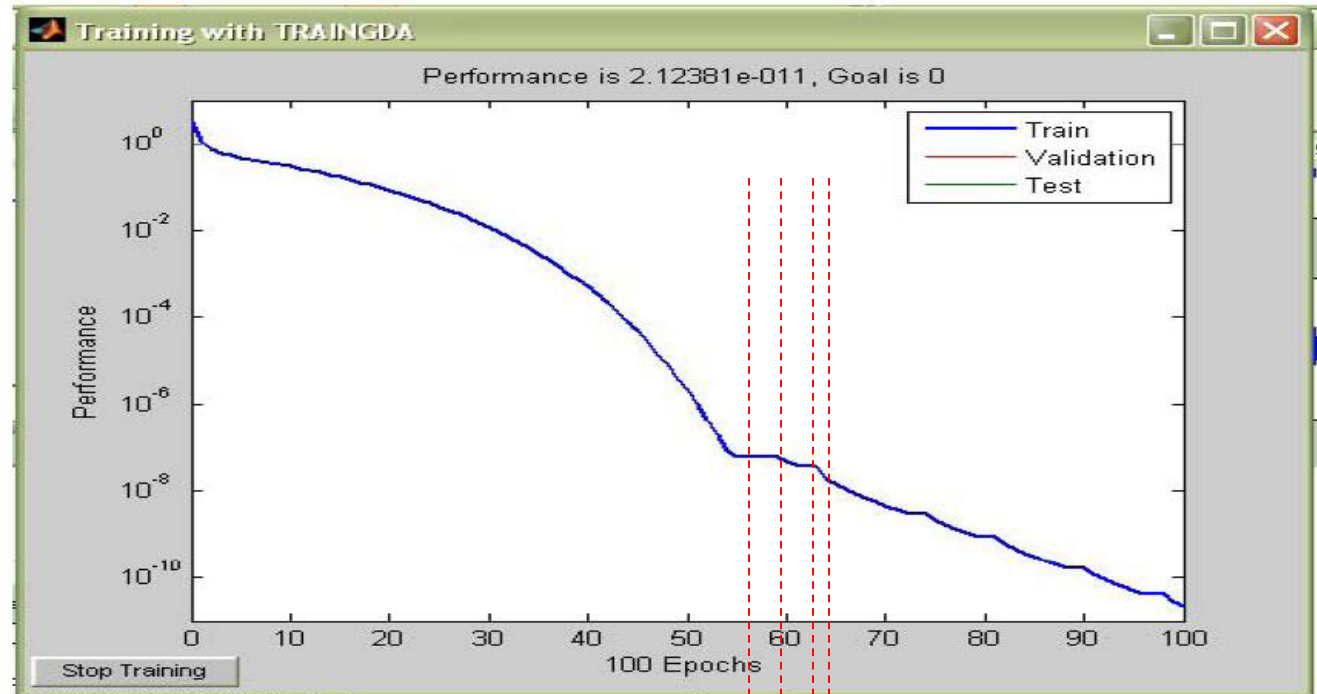
Large α could speed up the training process, but may cause instability.

To accelerate the convergence and yet avoid the danger of instability, we can apply two heuristics:

1. If the “sum of squared errors” is **decreasing** for several consequent epochs, then the learning rate parameter should be **increased** (typically by multiplying by 1.05).
2. If the “sum of squared errors” is **fluctuating, increasing, or becomes constant** for several consequent epochs, then the learning rate parameter should be **decreased** (typically by multiplying by 0.7).

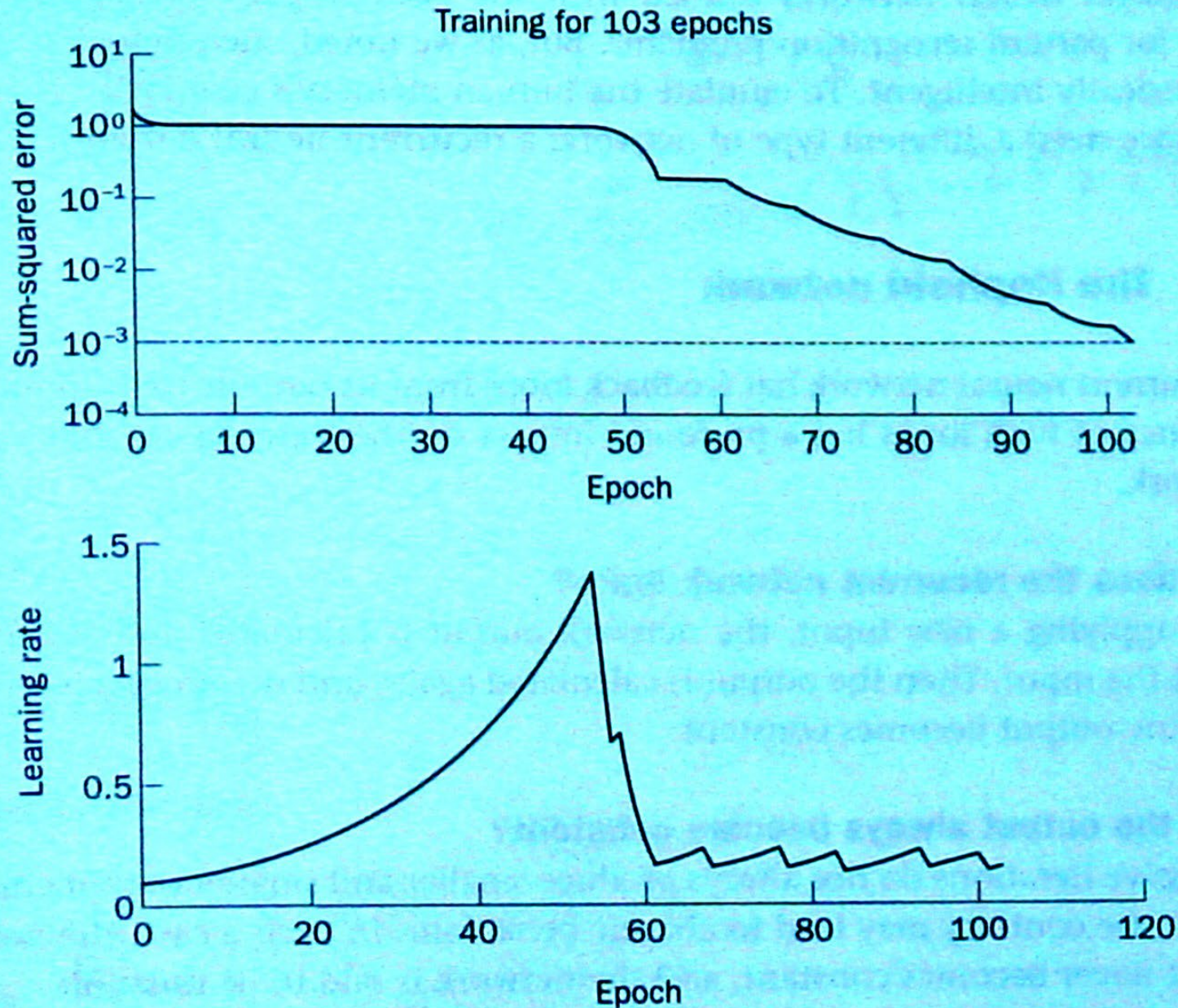
Example from BP training of XOR:

MATLAB-
NNtoolbox-
XOR.txt

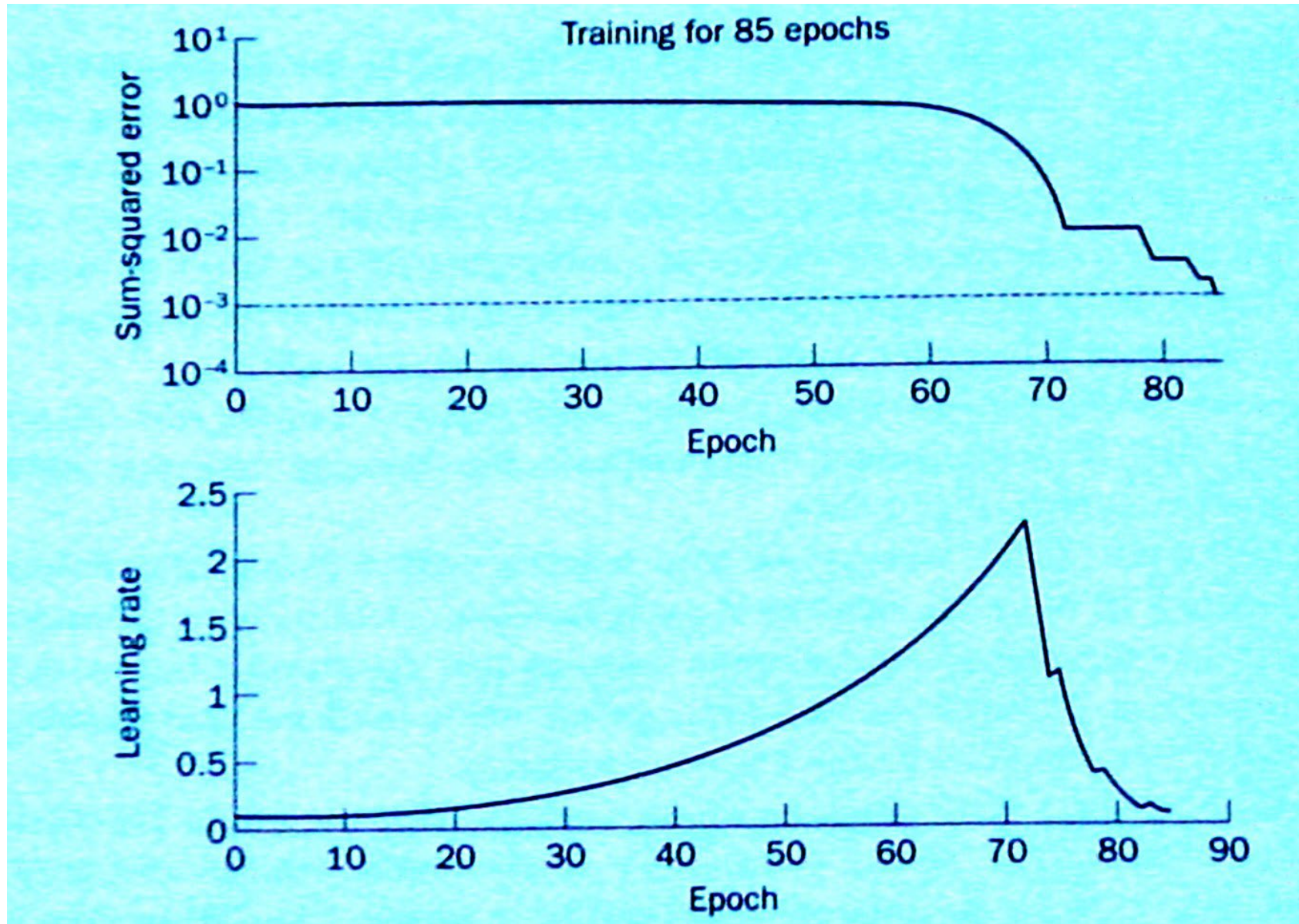


epochs	Learning rate	Error change	epochs	Learning rate	Error change
41	0.703999	-0.00017	61	0.368972	-7.04E-09
42	0.739199	-0.00012	62	0.38742	-2.29E-09
43	0.776159	-7.78E-05	63	0.406791	0
44	0.814967	-5.07E-05	64	0.284754	-1.83E-08
45	0.855715	-3.22E-05	65	0.298992	-5.71E-09
46	0.898501	-1.98E-05	66	0.313941	-2.90E-09
47	0.943426	-1.19E-05	67	0.329638	-2.00E-09
48	0.990597	-6.86E-06	68	0.34612	-1.57E-09
49	1.040127	-3.83E-06	69	0.363426	-1.28E-09
50	1.092133	-2.06E-06	70	0.381597	-1.05E-09
51	1.14674	-1.06E-06	71	0.400677	-8.30E-10
52	1.204077	-5.27E-07	72	0.420711	-5.93E-10
53	1.264281	-2.49E-07	73	0.441747	-2.65E-10
54	1.327495	-1.11E-07	74	0.463834	0
55	1.39387	-1.84E-08	75	0.324684	-7.41E-10
56	1.463563	0	76	0.340918	-4.52E-10
57	1.024494	0	77	0.357964	-3.19E-10
58	0.717146	0	78	0.375862	-2.34E-10
59	0.502002	0	79	0.394655	-1.59E-10
60	0.351401	-1.37E-08	80	0.414388	-6.89E-11

Learning with adaptive learning rate

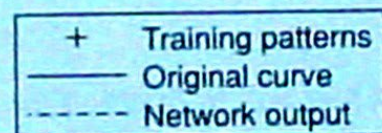
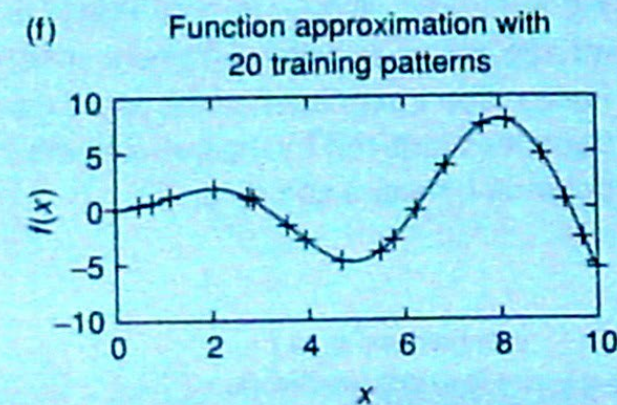
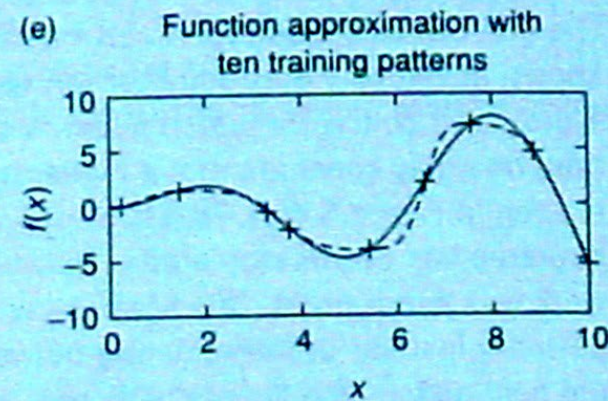
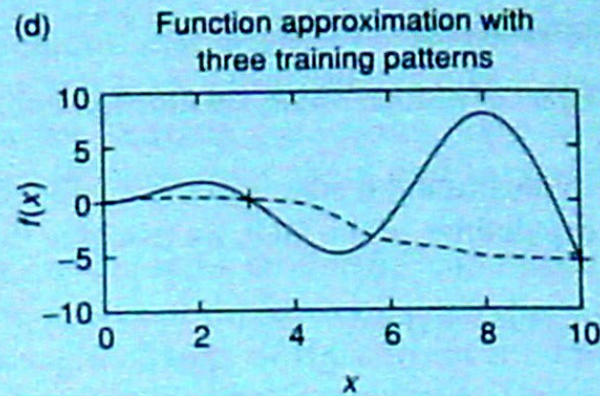
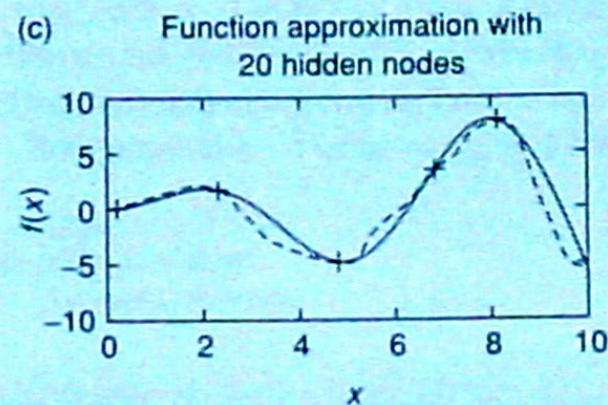
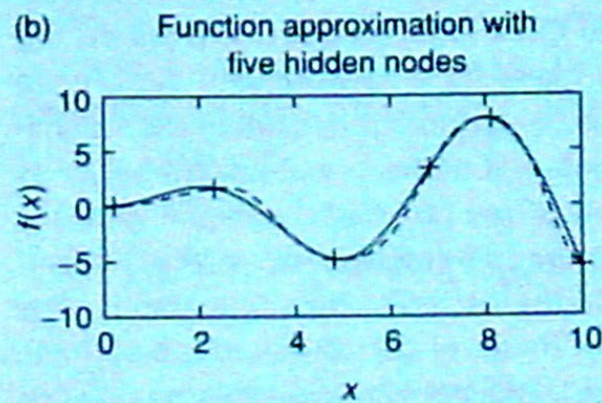
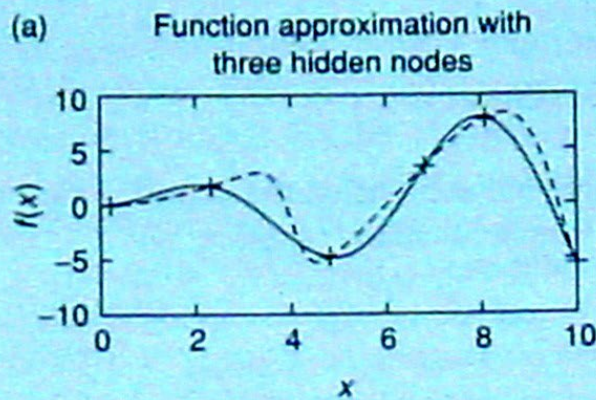


Learning with momentum and adaptive learning rate



Effect of hidden nodes on function approximation

- Example: **$f(x) = x \sin(x)$**
- Six input/output samples were selected in the range $[0:10]$.
- Figure (a): Three hidden nodes
- Figure (b): Five hidden nodes
- Figure (c): Twenty hidden nodes



(a),(b),(c) : Effect of the number of hidden nodes on MLP

approximation of $f(x)$.

(d),(e),(f) : Effect of the number of training patterns on MLP

approximation of $f(x)$. (**with *FIVE* hidden nodes**)

Comments on **effect of hidden nodes**

- **More hidden nodes is not always better.**
In this case, the network has over interpolated in between the samples.
- The network has **overtrained**. That is, the network starts to memorize the patterns instead of interpolating between them.
- For the case with three hidden nodes, the network's parameters were **not enough** to allow for adequate interpolation between samples.

Comments on **effect of training patterns**

- Five hidden nodes have been used in this case.
- Three samples (Figure (d)) were not enough to provide a good match with the original curve. They were not able to reproduce the relatively high nonlinearities of the function.
- A **higher number of training patterns** has provided better results (Figure (f)).

Overfitting and Underfitting

In machine learning, one challenge is ensuring your model generalizes well to unseen data.

Two common problems that can hinder generalization are overfitting and underfitting.

What is Underfitting?

Underfitting happens when your model is too simple to capture the underlying patterns in the training data.

[AI : Overfitting vs Underfitting\(ML\)-Part 16 | by Shankar Angadi | Aug, 2025 | Medium](https://medium.com/@angadi.saa/ai-overfitting-vs-underfitting-ml-part-16-e92c9eed14d8)

<https://medium.com/@angadi.saa/ai-overfitting-vs-underfitting-ml-part-16-e92c9eed14d8>

About your data

- We divide our data into 3 separate datasets: **training dataset, validation dataset and testing dataset.**
- A typical guideline would be 70% - 15% - 15% split.
- The training dataset is used to update model parameters.
- The test dataset is used for a final, unbiased evaluation of the fully trained model.
- The validation dataset serves as an independent set of data used during the model training process to monitor performance, tune hyperparameters, and prevent overfitting.

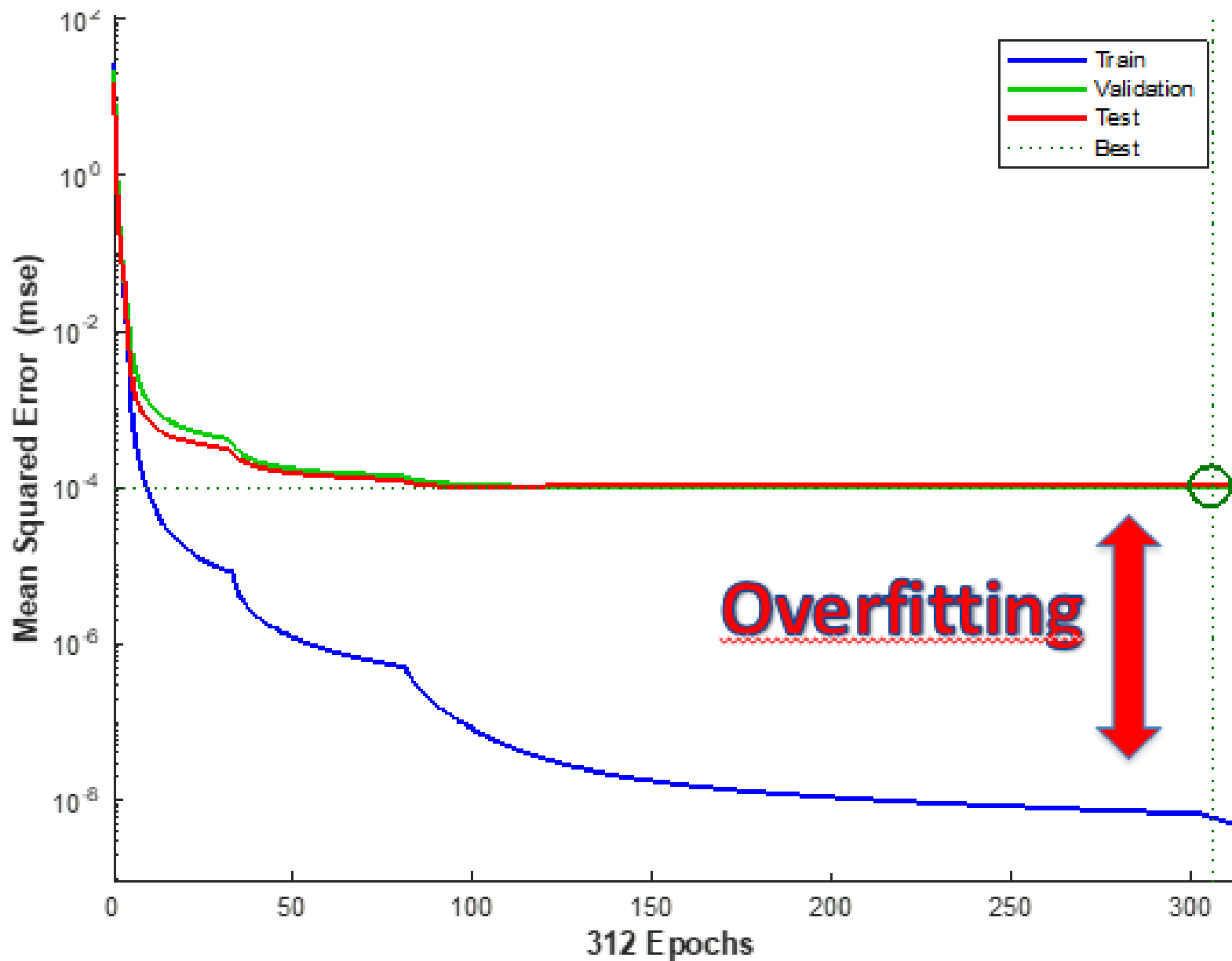
The use of the validation dataset

Hyperparameter Tuning – this helps in finding the best model architecture and settings. The commonly tuned hyperparameters are

- learning rate (it controls the step size in a gradient descent update on model parameters);
- batch size (the number of training samples processed before the model's weights are updated);
- Number of hidden layers and neurons (controls the model's complexity)
- Dropout rate
- Optimizer choice which is on choosing an algorithm to be used for training.

Overfitting Detection – see next figure. Overfitting is about excellent performance on the training set but poor generalization to unseen data.

Model Selection – When experimenting with different model architectures or algorithms, the validation set provides a basis for comparing their performance. The model that consistently performs best on the validation set is selected as the most promising candidate.



MSE curve shows Overfitting

Characteristics of Underfitting:

- Poor performance on both **training** and **test** data.
- **High bias** — model makes strong assumptions.
- Model fails to learn important trends in the data.

What is Overfitting?

Overfitting happens when your model is too complex, learning both the patterns and the noise in the training data.

Characteristics of Overfitting:

- Excellent performance on training data.
- Poor performance on test or unseen data.
- High variance — model is overly sensitive to small fluctuations.

Learning Curves Analysis

Underfitting (High Bias)



Good Fit (Balanced)



Overfitting (High Variance)



Learning Curves Interpretation

Underfitting: Both training and validation errors converge to a high value, indicating the model is too simple to capture the underlying patterns.

Good Fit: Both errors converge to a low value with minimal gap, showing the model generalizes well.

Overfitting: Large gap between training (low) and validation (high) errors, indicating the model memorizes training data but fails to generalize.

Data Fitting Analysis

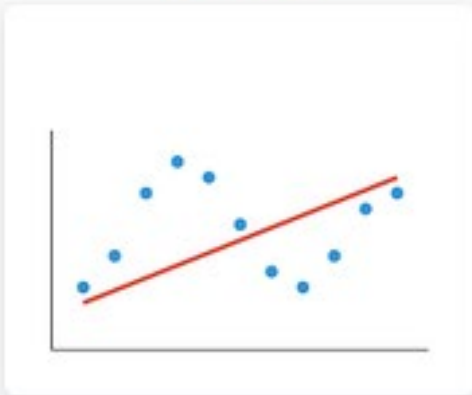
Blue dots: Training data points

Red/Green/Orange line: Model's learned function

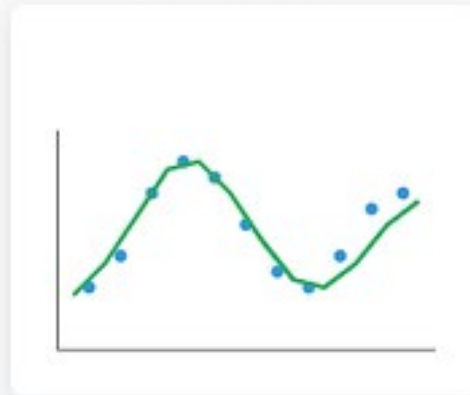
Notice how underfitting fails to capture the pattern, good fit captures the underlying trend, and overfitting memorizes noise with excessive complexity.

Data Fitting Examples

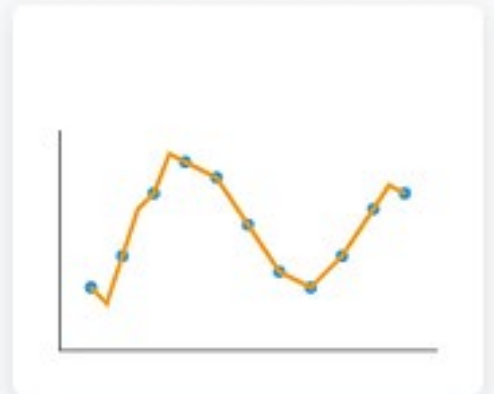
Underfitting: Linear Model on Non-linear Data



Good Fit: Appropriate Model Complexity



Overfitting: Too Complex Model



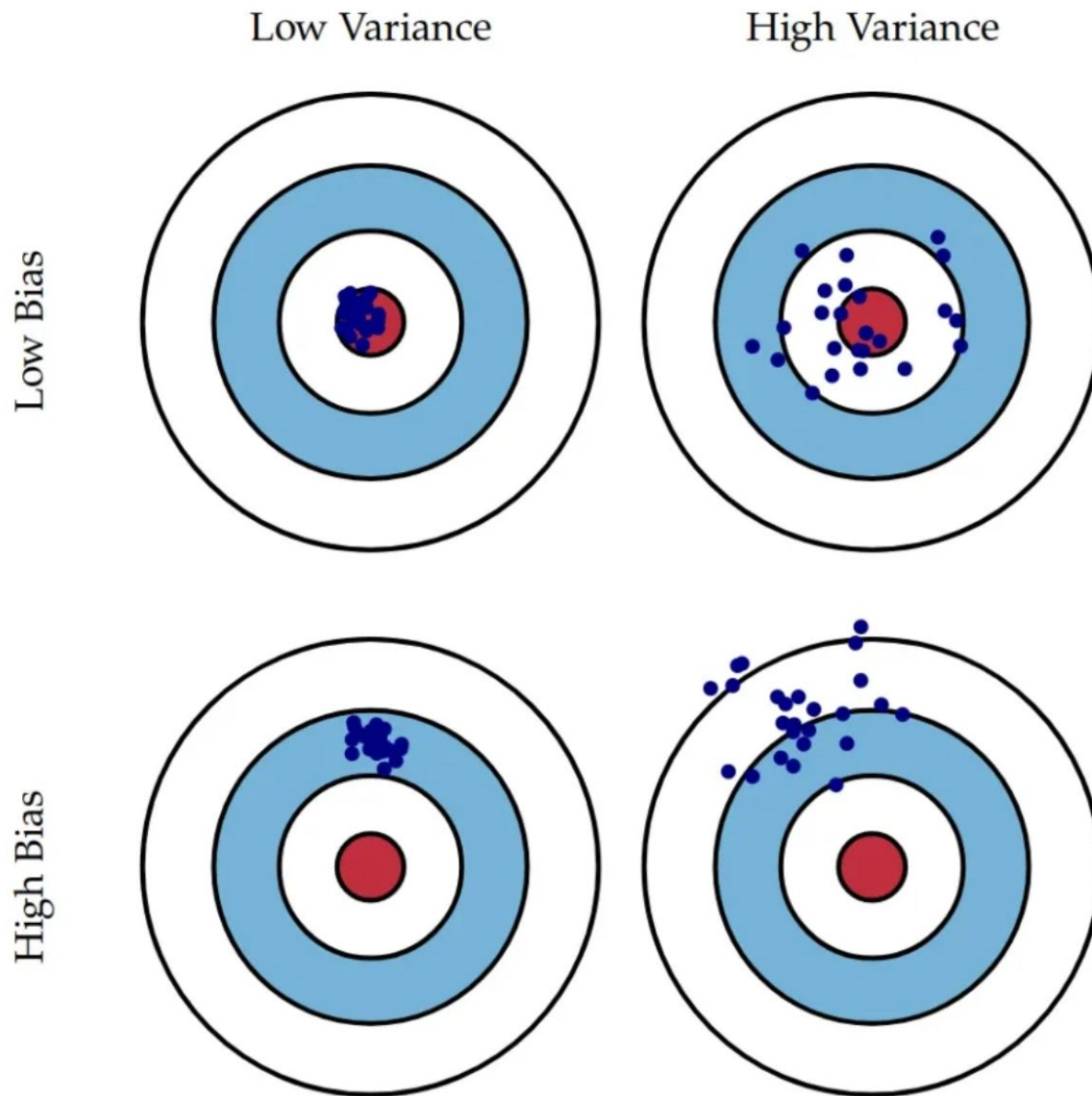


Fig. 1 Graphical illustration of bias and variance.

In machine learning model training, **variance** measures a model's sensitivity to the specific training data it encounters, indicating how much its predictions would change if trained on different subsets of the data.

High variance leads to overfitting, where a model becomes too complex and memorizes training data, including noise, resulting in poor performance on new, unseen data.

Key characteristics of high variance:

- **Overfitting:** The model performs very well on the training data but poorly on new data because it has "memorized" the training set's specifics, including noise.
- **Sensitivity to data:** The model's predictions change significantly when trained on slightly different datasets.
- **Complex models:** Complex models with many features or a high degree of flexibility are more prone to high variance.

How variance contributes to model performance:

- **High variance:** indicates that the model is too flexible, leading it to fit the noisy details of the training data rather than the true underlying patterns.
- A high-variance model has low bias, which allows it to capture complex patterns but also makes it prone to errors on new data.

Aspect	Underfitting	Good Fit	Overfitting
Training Error	High	Low-Medium	Very Low
Validation Error	High	Low-Medium	High
Model Complexity	Too Simple	Just Right	Too Complex
Bias	High	Balanced	Low
Variance	Low	Balanced	High
Generalization	Poor	Good	Poor
Solutions	Increase complexity, More features	Continue monitoring	Regularization, More data

Summary:

Methods to accelerate training:

1. Use higher learning rate
2. Use the momentum terms
3. Use adaptive learning rate

There are many python frameworks for implementing ANN for supervised learning. The Keras and MLPClassifier will be introduced next.

Dataset divided into training, validation and testing datasets. Overfitting: variance and bias